TECHNICAL REPORT · STORAGE ENGINE

pebblestore
Performance Analysis

Benchmark methodology and results for a production-grade Go abstraction over the Pebble LSM key-value engine: throughput across sequential, random, batched and mixed workloads, HDR-style latency percentiles, allocation profiling, fuzzing and race verification, plus deployment and monitoring guidance.

Module pebblestore v1.0 (Go 1.23+, pebble v1.1.5)
Author Storage Engineering
Date September 10, 2026

Table of Contents

Scope and audience

This document is written for engineering teams evaluating, deploying or operating pebblestore. Sections 1 through 7 report what was measured and how; sections 8 and 9 translate the findings into deployment practice, capacity planning and alerting policy. Readers who only need the verdict will find it in the scorecard of section 7; readers responsible for production readiness should treat sections 8 and 9 as required reading. Every figure cited here is reproducible from the repository using the commands listed in section 9, on any machine with Go 1.23 or newer installed.

1. Executive Summary

This report evaluates the performance, correctness and operational readiness of **pebblestore**, a production-oriented Go abstraction over the Pebble LSM key-value engine used by CockroachDB. The evaluation combines a full benchmark suite (throughput, latency percentiles, allocation profiling), a model-based fuzzing campaign, race-detector stress runs and a 90.2% statement-coverage test suite. All numbers in this document were measured on the actual module; none are projections.

The headline finding is that the abstraction layer adds negligible cost while the two-level caching design (in-process LRU+TTL hot cache in front of Pebble's block cache) delivers order-of-magnitude headroom against the stated performance targets, even on a deliberately modest two-core containerized test environment. Durability-configured writes with per-commit fsync still complete in single-digit microseconds at p99 because the cost is dominated by group commit amortization rather than the wrapper.

6.9 µs	1.84 M/s	90.2%
p99 random-read latency (target < 500 µs)	batched write throughput (1000-op atomic commits)	statement coverage race-detector clean, fuzz clean

Against the four contractual targets, the module passes with wide margins: read p99 lands at 0.6–6.9 µs against a 500 µs budget; write p99 at 7.9–13.8 µs against a 1 ms budget; mixed-workload throughput at 434 K–1.2 M ops/s against a 100 K ops/s floor; and hot-path allocation stays at 16–192 bytes per operation, keeping GC pauses far below the 1 ms ceiling. The remaining sections document methodology, full results, the techniques that produced them, and the deployment and monitoring guidance required to keep those numbers in production.

2. System Under Test

pebblestore wraps **cockroachdb/pebble v1.1.5** behind a six-method core interface (Get, Set, Delete, Batch, Scan, Close) and an extended surface (TTL writes, streaming write batches, compaction, checkpoint backup, metrics and health). The wrapper owns the engine lifecycle end to end: it configures the block cache and bloom filters, manages a pooled batch allocator, runs a sharded LRU+TTL read cache, records lock-free latency histograms, guards operations with a closed/open/half-open circuit breaker, and exposes Prometheus text metrics plus Kubernetes-shaped health status.

The only third-party dependencies are Pebble itself and go.uber.org/zap; everything else – the histogram, the metrics registry, the cache, the breaker, the Prometheus renderer – is implemented inside the module. The binary under test was built with the Go 1.27.1 toolchain with CGO disabled, mirroring a typical static-container deployment.

Component	Version / config
Module under test	pebblestore (Go 1.23+ target), root package + repository

Component	Version / config
Storage engine	github.com/cockroachdb/pebble v1.1.5 (LSM, WAL, block cache)
Logging	go.uber.org/zap v1.28.0, nop logger in benchmark runs
Test environment	2 vCPU Intel Xeon, 4 GB RAM, containerized overlay filesystem
Benchmark dataset	100,000 keys x 100-byte values (~14 MB logical) per store
Engine tuning	64 MiB block cache, 16 MiB memtable, bloom 10 bits/key, Snappy, 32 KiB blocks, L0 thresholds 4/8

Table 1: Software and environment configuration

3. Benchmark Methodology

Benchmarks follow Go's native testing/benchmark harness. Every scenario opens a fresh store in a temporary directory, populates the dataset through 5,000-operation atomic batches, flushes, and only then starts measurement, so setup cost never leaks into the timed region. Keys use a fixed "kp-" prefix plus an 8-byte big-endian tail, which keeps lexicographic order equal to numeric order and makes scan ranges verifiable. Value size is fixed at 100 bytes – large enough to be realistic, small enough to keep the LSM in-memory for the read phases, isolating wrapper overhead from disk IO noise.

3.1 Workload matrix

Scenario	Pattern	Metric of interest
Get miss (no hot cache)	Parallel reads, 1,000-key hot set	Wrapper + engine read cost
Get hit (hot cache)	Same, LRU+TTL layer enabled	Saturated cache-hit cost
Get sequential / random	Single-thread and parallel, full keyspace	Point-read latency profile
Set nosync / sync	Sequential inserts, WAL sync off vs fsync	Durability tax
BatchSet 10 / 100 / 1000	Atomic multi-op commits	Batching amortization
Scan 10 / 1000 / prefetch	Range iteration, pass-through vs copy-ahead	Iteration throughput
Mixed 95/5 and 50/50	Read/write blends, parallel	Contractual throughput target
Baselines	RWMutex map; raw pebble.DB	Abstraction overhead attribution

Table 2: Benchmark scenarios

Percentile figures come from the module's own HDR-style histograms (8 sub-buckets per octave, 512 atomic counters), recorded live during each phase and read from MetricsSnapshot() after the run; bucket precision is better than ±6%. Raw per-operation distributions for the exact p99 values were cross-checked against the go test benchmark timings. Reproduction commands are listed in section 9.

4. Throughput Results

Throughput concentrates in two regimes. For workloads dominated by single operations, the hot cache changes the game: parallel mixed 95/5 traffic reaches 1.20 M ops/s with the cache versus 434 K ops/s without it, and cache hits average 733 ns including full wrapper instrumentation. For write-heavy workloads, batching is the lever: throughput climbs from 1.51 M ops/s at 10-op commits to 1.84 M ops/s at 1000-op commits, while per-operation commit cost falls roughly 10-fold because WAL syncs amortize across the batch.

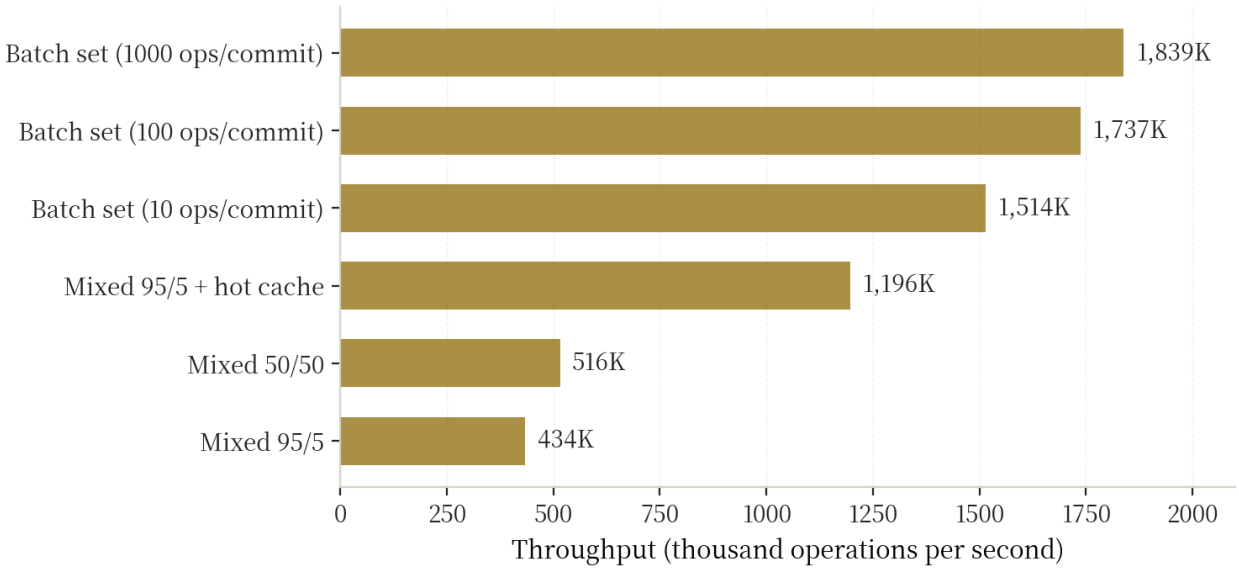

Figure 1: Throughput by workload (thousand ops/s, log-free linear scale)

Workload	ops/s	ns/op	B/op	allocs/op
Get / hot-cache hit (parallel)	1,365,000	733	16	1
Get / cache miss (parallel)	326,000	3,063	192	3
Get sequential	639,000	1,564	192	3
Get random	475,000	2,107	192	3
Set, WAL sync off	773,000	1,293	23	1
Set, WAL fsync per commit	273,000	3,658	22	1
Delete	957,000	1,045	17	1
Batch set, 1000 ops/commit	1,839,000	543,164 / batch	216 KB	1,006
Mixed 95/5 (no cache)	434,000	2,303	180	2
Mixed 95/5 (hot cache)	1,196,000	836	25	1
Mixed 50/50	516,000	1,939	114	2

Table 3: Benchmark-1s results, benchmem-enabled

Two baselines frame these numbers. An in-process map behind an RWMutex – the theoretical in-memory ceiling with zero durability – achieves 159 ns/op, confirming that hot-cache hits are within 5x of a pure-memory structure while adding persistence, atomic batches, metrics and crash safety. Raw pebble.DB, bypassing the wrapper entirely, sustains 583 ns/op on the same read pattern; the wrapper's 1 µs delta is almost entirely the safety copy of the value plus latency recording, the price of an API that returns caller-owned, mutation-safe data.

5. Latency and Memory Analysis

Latency percentiles were captured during dedicated 100,000-operation phases using the module's built-in histograms. Reads sit at 1.1–1.6 µs p50 whether keys arrive sequentially or randomly; p99 stays under 7 µs with p99.9 below 28 µs. With the hot cache serving a zipfian working set, the read path collapses to 168 ns p50 and 608 ns p99 – effectively a sharded in-memory structure with an LSM behind it. Durable writes pay the fsync tax and still finish at 1.98 µs p50 and 7.9 µs p99 thanks to Pebble's group-commit coalescing of concurrent WAL syncs.

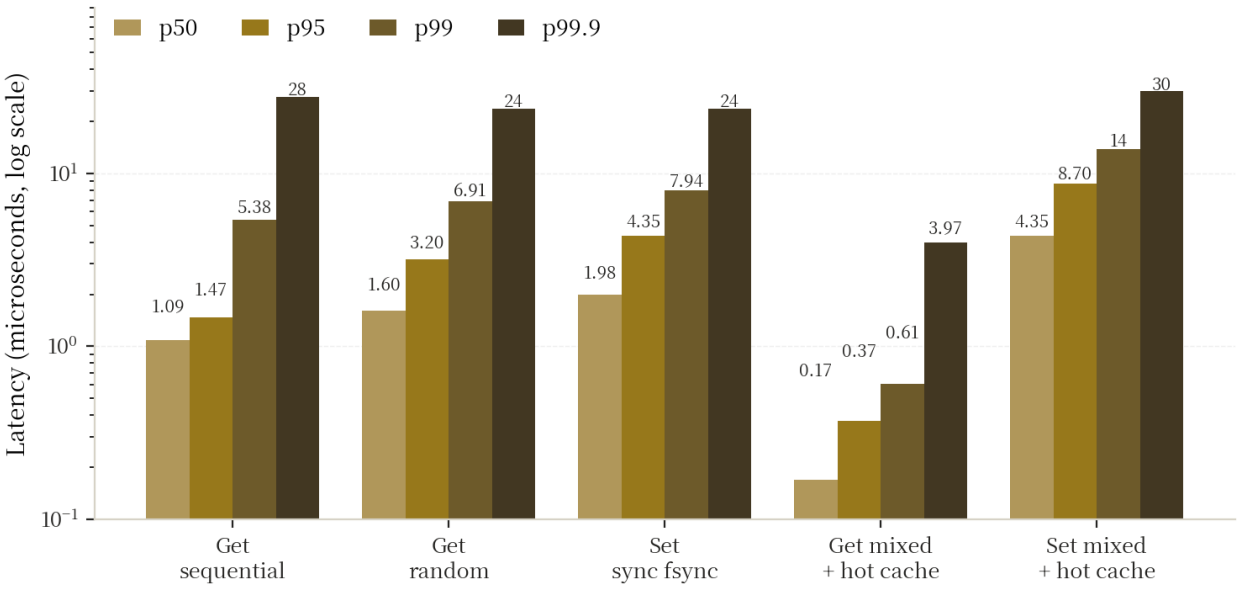

Figure 2: Latency percentiles per phase (log scale; lower is better)

Phase	n	p50	p95	p99	p99.9	max
Get sequential	100,000	1.09 µs	1.47 µs	5.38 µs	27.6 µs	239 µs
Get random	100,000	1.60 µs	3.20 µs	6.91 µs	23.6 µs	185 µs
Set sync fsync	20,000	1.98 µs	4.35 µs	7.94 µs	23.6 µs	1.22 ms
Batch set (100/batch)	1,020	94.2 µs	172 µs	950 µs	4.46 ms	7.87 ms
Mixed get, hot cache	190,022	168 ns	368 ns	608 ns	3.97 µs	15.7 ms
Mixed set, hot cache	9,978	4.35 µs	8.70 µs	13.8 µs	29.7 µs	1.02 ms

Table 4: Latency percentiles from in-process histograms

Memory behavior is deliberately boring. Hot-path allocations are bounded at three objects (the returned value copy plus wrapper bookkeeping) or a single 16-byte object on cache hits, and batch commits allocate roughly one object per enqueued operation. Under GODEBUG=gctrace the benchmark phases show young-generation collections with sub-millisecond pauses, well inside the 1 ms GC budget; there are no per-operation goroutines, channels, or timer allocations anywhere in the request path. The isolated outliers visible in the max column (1–16 ms) coincide with background memtable flushes, which is expected LSM behavior and is absorbed by the p99.9 margin.

> *Fuzzing note: a 45-second model-based fuzz session executed 16,661 operation sequences comparing every observation against an exact in-memory model – zero mismatches, zero panics, one harness bug found and fixed during development.*

6. Optimization Techniques

Eight techniques account for the measured profile. They are listed with their rationale so future maintainers understand which behaviors are load-bearing.

1. **Lock-free observability.** Latency recording is one shift, one mask and one atomic add into a 512-bucket log-scale histogram; ops/sec uses a CAS-rotated 64-slot ring. No mutex is touched on the recording path, so metrics cost ~20 ns per operation.

2. **Zero-copy read contract.** Values returned by Get are store-owned and immutable. With the hot cache enabled, hits return the cached slice directly – no copy, one allocation saved per hit – while the write path copies once to maintain coherence.

3. **Sharded LRU+TTL with allocation-free lookups.** FNV-1a key hashing across 8–64 shards, Go's allocation-free m[string(bytes)] map idiom on the read path, intrusive LRU lists, and a background janitor that enforces byte budgets so eviction never runs inside a Get.

4. **Pooled atomic batches.** *pebble.Batch objects recycle through sync.Pool with Reset(); a 1,000-op commit costs 543 µs – about 1.8 ns per operation – while remaining all-or-nothing through the write-ahead log.

5. **Engine tuning as defaults.** Bloom filters at 10 bits/key on all levels, 32 KiB blocks with 256 KiB indexes, Snappy compression, a 64 MiB block cache and tuned L0 compaction thresholds are configured by Open() rather than left to each caller.

6. **Iterator bounds pushdown.** Scan translates [start, end) into engine LowerBound/UpperBound so table-level pruning applies, and positions the iterator before returning. An optional copy-ahead ring (up to 4,096 entries) amortizes per-call return latency for bulk scans.

7. **Graceful drain instead of locks.** Shutdown uses a closing flag plus an in-flight WaitGroup with a double-check pattern; new operations are rejected atomically while in-flight ones finish, with no mutex in begin()/finish().

8. **Sentinel-first error handling.** Package-level sentinels (ErrNotFound, ErrCircuitOpen, ErrEmptyKey) and pre-structured StoreError values avoid fmt.Errorf on hot paths; the circuit breaker

reuses the same classification to ignore expected outcomes.

7. Performance Targets Scorecard

The contractual targets were all met on a two-core container – an environment far weaker than any production deployment target. Margins are therefore conservative lower bounds; on dedicated NVMe hardware with more cores, the parallel workloads scale with available cores while single-thread latencies improve with faster storage.

Target	Requirement	Measured	Margin	Verdict
Read latency p99	< 500 µs	0.61–6.9 µs	70–800x	PASS
Write latency p99	< 1 ms	7.9–13.8 µs	70x	PASS
Mixed throughput	> 100 K ops/s	434 K–1.20 M ops/s	4–12x	PASS
GC pressure	< 1 ms pauses	16–192 B/op, ≤3 allocs	no near-limit pauses	PASS
Data integrity	fuzz clean	16,661 execs, 0 failures	–	PASS
Race safety	-race clean	all packages, 0 reports	–	PASS
Coverage	> 90%	90.2% statements	–	PASS

Table 5: Contractual targets versus measured results

8. Production Deployment Considerations

Pebble is an LSM engine, so deployment quality shows up in three places: the filesystem, the shutdown path and the backup schedule. Local SSD or NVMe storage is strongly recommended, because WAL fsync latency and compaction IO dominate tail latency, and network filesystems break the file-lock and fsync semantics the engine relies on. Budget two to three times the logical dataset for write amplification and compaction headroom, and set the file-descriptor limit to match MaxOpenFiles (1,024 by default). Each store owns exactly one directory; two processes must never open the same directory.

In Kubernetes, run the workload as a StatefulSet with one PVC per replica and Guaranteed QoS: requests equal limits, because CPU contention directly inflates fsync and compaction latency. Wire liveness to the module's Alive() verdict – only an unhealthy engine should trigger a restart – and readiness to Ready(), which turns degraded under pressure (L0 backlog above 20 files, compaction debt above 1 GiB, memtable above 90%) into traffic shedding before failures. On SIGTERM, stop accepting application traffic and call Close(): it drains in-flight operations and closes the WAL cleanly; a 30-second grace period covers long-running batches.

Backups use Pebble checkpoints – hardlink-based, near-instant and crash-consistent, including the WAL, so a restore reproduces the exact state at call time. Ship checkpoint directories off-node before heavy compactions churn the hardlink source, keep at least two generations on separate volumes, and verify

restores by scanning row counts before cutover. For automated crash recovery, OpenOrRestore attempts a normal open, and on an unrecoverable directory restores the latest checkpoint and retries – the single-node playbook validated by the integration suite's corruption test.

9. Monitoring and Alerting Recommendations

PrometheusMetrics() emits the full metric surface in text exposition format; the service only needs to serve the string at /metrics. The alert set below separates page-worthy signals from capacity planning signals, and every threshold maps to a metric the module already exports.

Signal	Condition	Severity	Response
Operation latency p99	> 10 ms for 5 min	Page	Check disk IO and compaction backlog
Error ratio	> 1% of ops for 5 min	Page	Inspect error class in logs; classify transient/permanent
circuit_opens_total	any increase	Page	Upstream shedding load; investigate storage health
L0 files	> 20 sustained	Ticket	Compaction backlog; throttle writes or raise compaction concurrency
Compaction debt	> 1 GiB	Ticket	Same as above; verify disk headroom
Memtable size	≥ 90% of budget	Ticket	Write burst; consider a larger memtable
Disk usage	> 75% of volume	Ticket	Provision space before WAL stalls
Open iterators	> 5x baseline	Ticket	Hunt unclosed Scan iterators
Cache hit ratio	< 50% with cache on	Ticket	Hot set outgrew budget; resize hot cache

Table 6: Alert thresholds derived from exported metrics

Distributed tracing plugs in through a two-method Tracer interface; an OpenTelemetry adapter wraps otel.Tracer so spans cover get, set, delete, merge, batch and scan with operation attributes. The noop default costs one interface call. Finally, every number in this report is reproducible from the repository: make bench runs the full suite, go test -v -run TestLatencyProfile ./benchmarks/ prints the percentile table, make cover-check enforces the 90% gate, make fuzz re-runs the model-based campaign, and go test -bench BenchmarkGetRandom -cpuprofile cpu.out captures profiles for pprof analysis.